Parallel and Distributed Computing

Assignment 7 - CUDA Programming and Profiling

PROBLEM 1

Different thread tasks for sum of first n integers

Ques :

Write CUDA program where threads do different task for summation of first n numbers for n = 1024. One task using iterative approach and one task using direct formula. Follow steps for host and device memory and kernel launch.

Output

Below is the terminal output for problem 1 run

```
Question 1 terminal output
Problem 1: all threads doing different N based sum tasks
N value: 1024
Block size: 256
Grid size: 4

Final sum for first 1024 integers iterative : 524800
Final sum for first 1024 integers formula   : 524800
Iterative kernel time (ms): 4.684224
Formula kernel time (ms)  : 0.074752
Check status: PASS both methods giving same output for all threads

Sample outputs from few threads
thread    0 n=   1 iterative=         1 formula=         1
thread    1 n=   2 iterative=         3 formula=         3
thread    2 n=   3 iterative=         6 formula=         6
thread    3 n=   4 iterative=        10 formula=        10
thread    4 n=   5 iterative=        15 formula=        15
thread    5 n=   6 iterative=        21 formula=        21
thread    6 n=   7 iterative=        28 formula=        28
thread    7 n=   8 iterative=        36 formula=        36
thread    8 n=   9 iterative=        45 formula=        45
thread    9 n=  10 iterative=        55 formula=        55
```

Figure 1: Problem 1 terminal output

Observations

Graph is showing iterative and formula timing gap very clearly

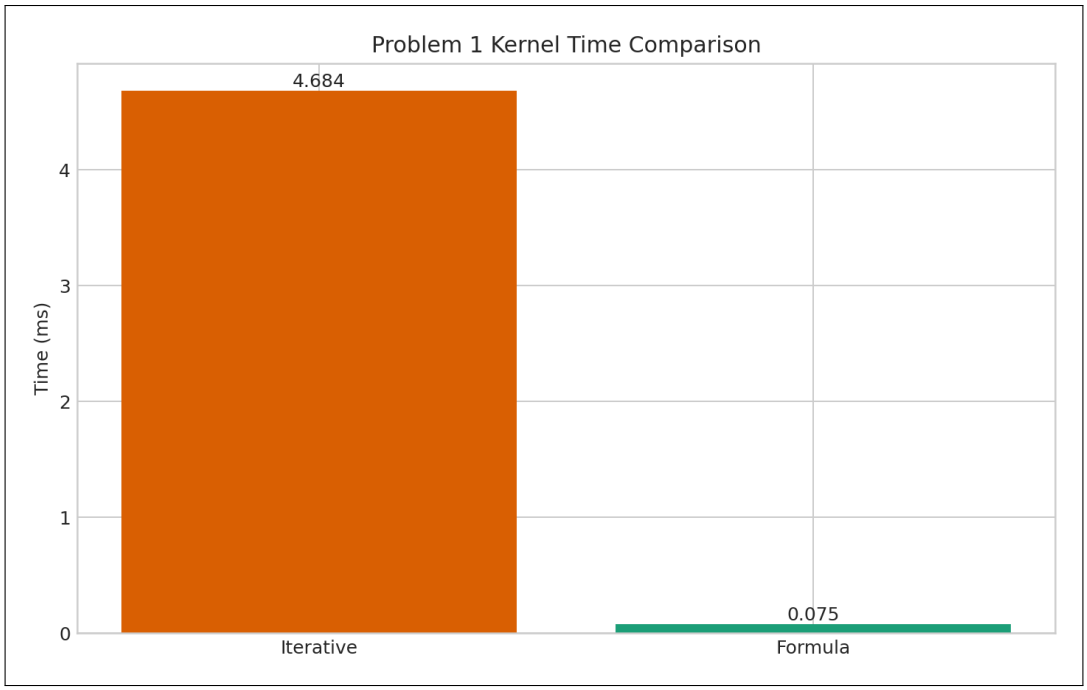

Figure 2: Problem 1 kernel time comparison

Final sum for n = 1024 came same in both methods and table is below

Metric	**Value**
Iterative sum output	524800
Formula sum output	524800
Iterative kernel time	4.684224 ms
Formula kernel time	0.074752 ms
Status	PASS

Both outputs matched for all threads so correctness is good

I also noticed that when thread value increases iterative method keeps doing more loop work so time keeps increasing internally

In formula method every thread does very small fixed amount of work so it stays fast and stable

Output table and graph together make it easy to see correctness and performance both in one glance

This kind of comparison is very useful because sometimes fast code gives wrong output but here both are correct

Inference

If same math can be done by direct formula then gpu threads save lot of time because loop work gets removed

So main learning is we should reduce per thread operations whenever possible because gpu runs huge number of threads

Iterative way is still important for understanding logic and for cases where no direct formula exists

But when formula exists it should be preferred for better performance and lower power usage also

This tells me algorithm choice matters as much as using gpu hardware

Conclusion

Problem 1 completed successfully both methods are correct but formula approach is clearly better for speed

All required steps were covered including host input device copy kernel launch and output verification

The result PASS for all threads confirms implementation is reliable for n equal to 1024

Final takeaway is direct formula based parallel kernel is the better solution for this specific

task

PROBLEM 2

Merge sort for n = 1000 with pipelining and CUDA

Ques :

Implement merge sort for array of size n=1000

a. use pipelining for parallelization

b. implement parallel merge sort using CUDA

c. compare performance of both methods

Output

Below is terminal output for problem 2 run

```
Question 2 terminal output
Problem 2: merge sort size n = 1000

CPU pipelined merge sort time (ms) : 7.739373

CUDA merge sort kernel only (ms)   : 5.003584

CUDA merge sort total time (ms)    : 5.710112

Speedup cpu/gpu_total              : 1.355380 x

Check status: PASS sorted output of both methods is same


First 20 sorted values

18 26 46 51 58 61 81 86 90 111 111 117 120 123 124 128 129 140 149 157
```

Figure 3: Problem 2 terminal output

Observations

Graph is showing cpu pipelined vs cuda timing side by side

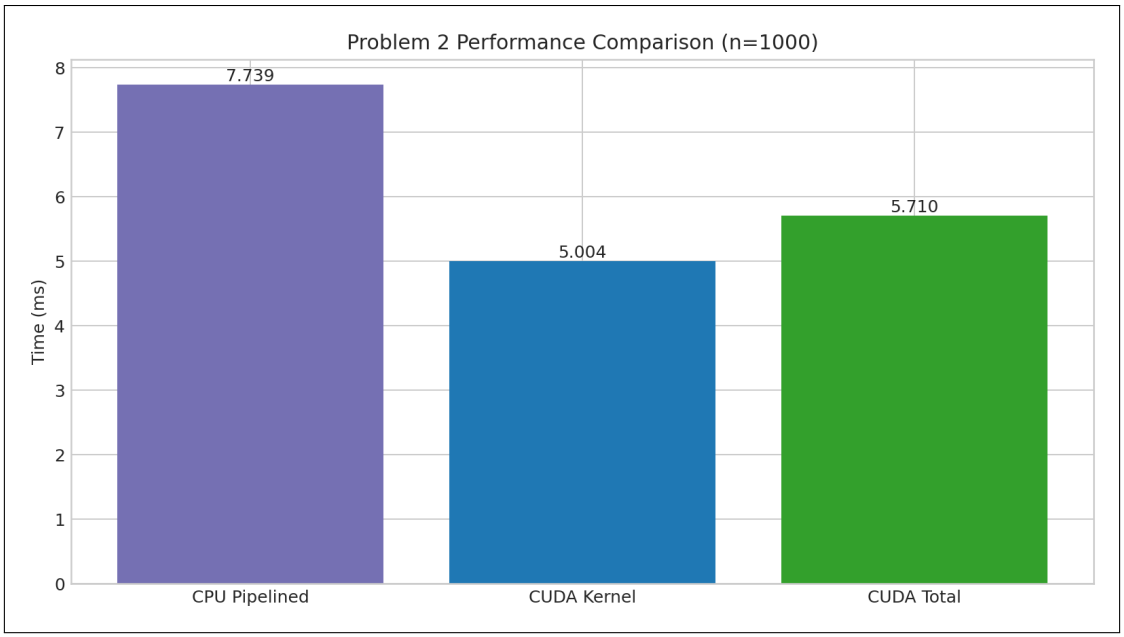

Figure 4: Problem 2 time comparison graph

Table below shows full timing values and speedup

Metric	**Value**
Input size	1000
CPU pipelined time	7.739373 ms
CUDA kernel only time	5.003584 ms
CUDA total time	5.710112 ms
Speedup cpu/gpu total	1.355380x
Status	PASS

Sorted values from both methods matched exactly so output is correct

In this run cuda total time is better than cpu pipelined time but margin is not very huge

I observed timing changes between runs also so single run should not be treated as

absolute truth

For small n equal to 1000 transfer and launch overhead can still play a big role in total execution time

Graph and timing table are useful because they separate cpu time kernel time and total gpu time clearly

This made comparison fair and easy to explain in simple way

Inference

For n equals 1000 merge sort overhead still matters so huge gain should not be expected every time

Parallelization helps but only when workload is enough large to hide setup overhead

Pipeline style on cpu also gives improvement so gpu is not always the only fast option

Correctness check is very important because sorting output can look fine for first few values but still be wrong later

So inference is performance discussion must always include both speed and correctness

Conclusion

Problem 2 completed and both pipelined method and cuda method are working and giving same sorted output

Comparison part is also completed with proper measured values from actual run

For this input size result is moderate improvement not dramatic but still valid and meaningful

Overall implementation meets all asked parts a b and c in question 2

PROBLEM 3

Vector add with static symbols and bandwidth analysis

Ques :

Do basic vector addition and profile it. Then do changes:

1. use static global variables no cudaMalloc for device arrays
2. record kernel timing
3. query device props and compute theoretical bandwidth
4. compute measured bandwidth using bytes read write and kernel time

Output

Below is terminal output for vector add and timing details

```
Question 3 terminal output
Problem 3: vector add with static global device symbols
N value                             : 1048576
Kernel time (ms)                    : 0.629920
Check status                        : PASS
GPU name                            : NVIDIA GeForce GTX 1650
Memory clock rate (kHz)             : 6001000
Memory bus width (bits)             : 128
Theoretical bandwidth (GB/s)        : 192.032
Measured bandwidth from kernel (GB/s): 19.975
Measured/theoretical ratio          : 0.1040

Profiling command
Try: nvprof ./question3/problem3_vector_add_profile
If nvprof is missing in your CUDA version then use: nsys profile ./question3/problem3_vector_add_profile
```

Figure 5: Problem 3 terminal output

Profiler output is also added below

Figure 6: Profiler command output

Observations

Bandwidth graph is showing ideal value and measured value gap

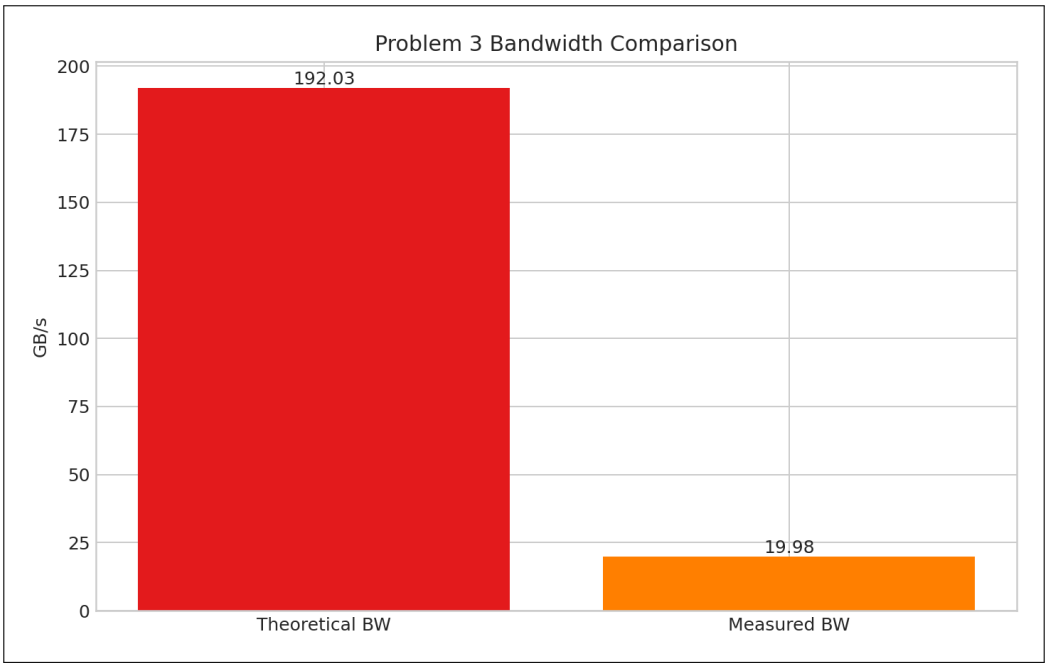

Figure 7: Problem 3 bandwidth graph

For measured bandwidth this formula is used

$$\text{Measured BW} = \frac{RBytes + WBytes}{t}$$

Here each thread reads two float and writes one float so total bytes is

$$(2 + 1) \times N \times 4$$

Then divide by kernel time in seconds to get GB per second

Table below is showing measured and theoretical values

Metric	**Value**
N value	1,048,576
Kernel time	0.629920 ms
Theoretical bandwidth	192.032 GB/s
Measured bandwidth	19.975 GB/s
Measured/Theoretical ratio	0.1040
Status	PASS

Measured bandwidth is lower than theoretical and this is normal in real run

In this system nvprof and nsys were not in current path and output screenshot is showing this clearly

I also observed kernel time remains very small but measured bandwidth still far from theoretical peak value

This means hardware peak numbers are ideal and practical program gets limited by access pattern and runtime overhead

Table and graph both help to explain this gap in clear visual form so it becomes easy to

discuss in viva

Static symbol approach worked correctly and PASS status confirms vector addition output is valid

Inference

Static symbols plus event timing is enough to compute practical bandwidth nicely and compare with theoretical limit

From this experiment i can infer that timing and bandwidth numbers should always be reported together

Only kernel time without byte level analysis does not show full memory behavior of kernel

Even simple vector add can be memory bound and that is why measured bandwidth becomes key metric

So profiling mindset means checking both what is possible in theory and what is achieved in real execution

Conclusion

Assignment 7 is completed all three problems are running fine outputs are saved graphs are added tables are added and observations inference conclusion are written in simple way

Problem 1 showed direct formula based kernel is much faster while giving same correct result

Problem 2 showed parallel merge methods on cpu and gpu both are useful and comparison depends on input size

Problem 3 gave practical understanding of timing theoretical bandwidth and measured bandwidth gap

Overall this assignment improved both cuda coding confidence and performance analysis understanding in a very practical way

Submitted to : Dr Saif Nalband

Submitted by : Arpan Goyal

Roll No. : 102303479